Node.js

Node.js

- **Node.js is an open-source and cross-platform JavaScript runtime environment.**
- **It is a powerful tool suitable for a wide range of projects.**
- **Node.js stands out as a game-changer.**
- **Imagine using the power of JavaScript not only in your browser but also on the server side.**

What is Node.JS?

1. **JavaScript Runtime: Node.js runs on the V8 JavaScript engine, which is also the core engine behind Google Chrome. However, unlike the browser context, Node.js executes JavaScript code outside of the browser.**

2. **Single Process Model: A Node.js application operates within a single process. This design choice contributes to Node.js' performance.**

3. **Asynchronous I/O: Node.js provides a set of asynchronous I/O primitives in its standard library. These primitives prevent JavaScript code from blocking, making non-blocking behavior the norm. When performing I/O operations (e.g., reading from the network, accessing databases, or the filesystem), Node.js doesn't waste CPU cycles waiting. Instead, it resumes operations when the response arrives.**

What is Node.JS?

4. **Concurrency Handling: Node.js efficiently handles thousands of concurrent connections using a single server. It avoids the complexities of managing thread concurrency, which can lead to bugs.**

5. **JavaScript Everywhere: Frontend developers familiar with JavaScript can seamlessly transition to writing server-side code using Node.js. You don’t need to learn a different language.**

6. **ECMAScript Standards: Node.js supports the latest ECMAScript standards. You can choose the version you want to use, independent of users’ browser updates.**

How Node.JS Works?

- Node.js accepts the request from the clients and sends the response, while working with the request node.js handles them with a single thread.
- To operate I/O operations or requests node.js use the concept of threads.
- Thread is a sequence of instructions that the server needs to perform.
- It runs parallel on the server to provide the information to multiple clients.
- Node.js is an event loop single-threaded language.
- It can handle concurrent requests with a single thread without blocking it for one request.

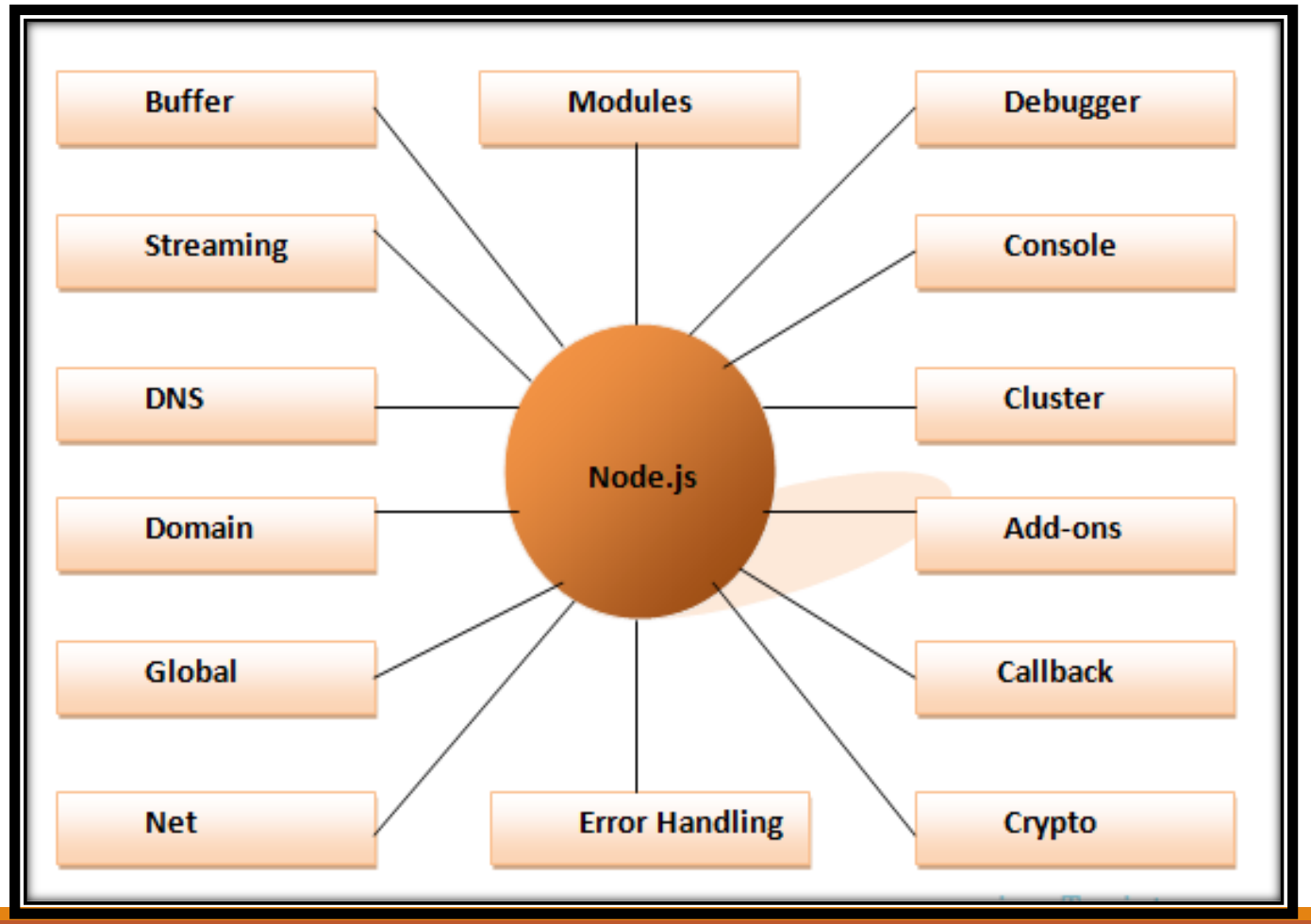
Why Node.JS?

- Easy to Get Started: Node.js is beginner-friendly and ideal for prototyping and agile development.
- Scalability: It scales both horizontally and vertically.
- Real-Time Web Apps: Node.js excels in real-time synchronization.
- Fast Suite: It handles operations quickly (e.g., database access, network connections).
- Unified Language: JavaScript everywhere—frontend and backend.
- Rich Ecosystem: Node.js boasts a large open-source library and supports asynchronous, non-blocking programming.

Application of Node.JS

- Real-time chats
- Complex single-page applications
- Real-time collaboration tools
- Streaming apps
- JSON APIs

Different Parts of Node.JS



Installation of Node.JS

- **https://nodejs.org/en/download**

Console based Application

hello.js

```
console.log("hello");
```

- Open **Node.js Command prompt**
- Go to the path of directory where the program is stored.

C:\Users\dyp> cd Desktop

C:\Users\dyp\Desktop> **node hello.js**

Web based Application

demo.js

```
var http = require('http'); // Require http header
http.createServer(function (req, res) {      // Create server
res.writeHead(200, {'Content-Type': 'text/html'}); // HTTP Status: 200 : OK,
                                                   // Content Type: text/html
 // Send the response body as "Hello World!"
    res.end('Hello World!');
  }).listen(8080);
console.log('Server running at http://127.0.0.1:8080/');    // Console will print the message
```

Web based Application

demo.js

```
http = require('http');

listener = function (request, response) {

  // Send the HTTP header, HTTP Status: 200 : OK, Content Type: text/html

response.writeHead(200, {'Content-Type': 'text/html'});

response.end('Hello World’); // Send the response body as "Hello World"

};

server = http.createServer(listener);

server.listen(8080);

console.log('Server running at http://127.0.0.1:8080/'); // Console will print the message
```

Web based Application

- Open **Node.js Command prompt**
- Go to the path of directory where the program is stored.

  C:\Users\dyp> cd Desktop

  C:\Users\dyp\Desktop> **node demo.js**

- Open **browser and give** http://127.0.0.1:8080 **or** http://localhost:8080

Node.js REPL

- REPL (**READ, EVAL, PRINT, LOOP**) is a computer environment similar to Shell (Unix/Linux) and command prompt.
- Node comes with the REPL environment when it is installed.
- System interacts with the user through outputs of commands/expressions used.
- It is useful in writing and debugging the codes.

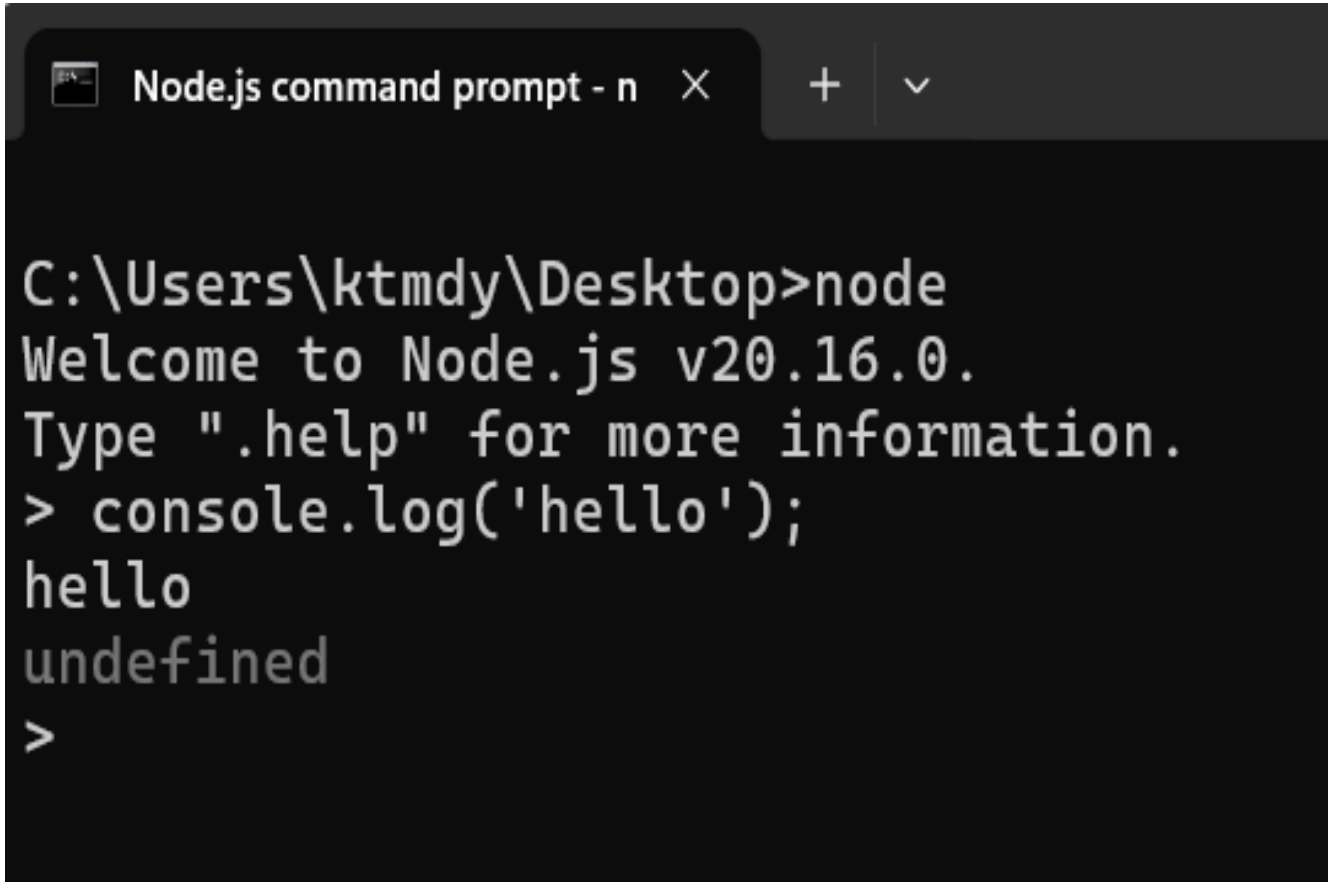
Node.js REPL

- **Read:** It reads the inputs from users and parses it into JavaScript data structure. It is then stored to memory.
- **Eval:** The parsed JavaScript data structure is evaluated for the results.
- **Print:** The result is printed after the evaluation.
- **Loop:** Loops the input command.
- To come out of NODE REPL, press ctrl+c twice

Node.js REPL

- Open node.js command prompt

C:\Users\ktmdy\Desktop>node



NPM

- NPM − an acronym for Node Package Manager, refers to the Command line utility to install Node.js packages, perform version management and dependency management of Node.js packages.
- It also provides an online repository for node.js packages/modules which are searchable on https://www.npmjs.com/.
- A detailed documentation of NPM commands is also available on this link.

NPM

- **Install Package Locally**

npm install <Module Name>

- **For example, following is the command to install a famous Node.js web framework module called express –**

npm install express

- **Now you can use this module in your js file as following –**

```
var express = require('express');
```

Node.js Callbacks

- Callback is an asynchronous equivalent for a function.
- It is called at the completion of each task.
- In Node.js, callbacks are generally used.
- All APIs of Node are written in a way to supports callbacks.
- For example: when a function start reading file, it returns the control to execution environment immediately so that the next instruction can be executed.

Node.js Callbacks

- In Node.js, once file I/O is complete, it will call the callback function.
- So there is no blocking or wait for File I/O.
- This makes Node.js highly scalable, as it can process high number of request without waiting for any function to return result.

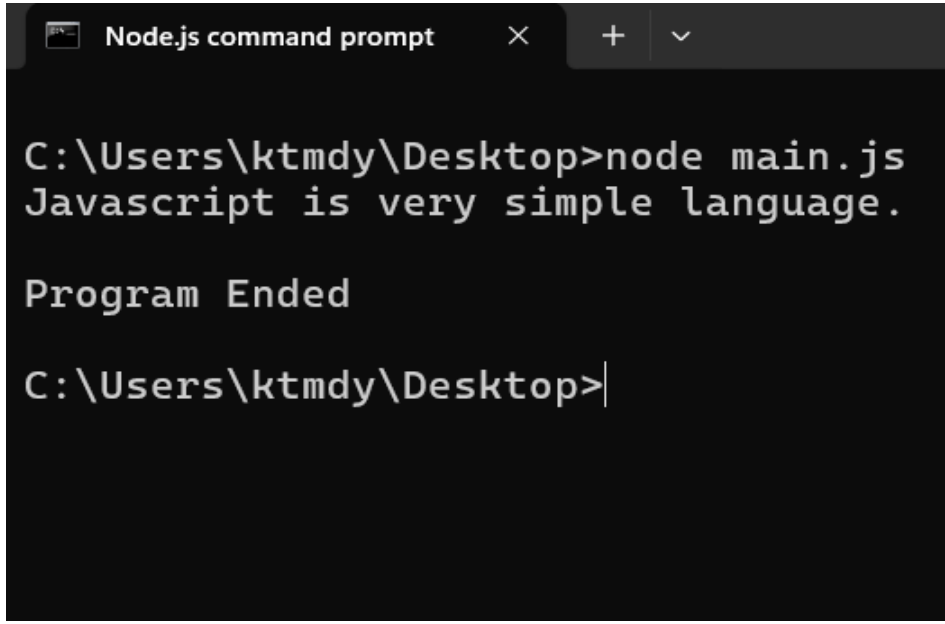
Blocking Code without callbacks

Create a text file named **input.txt** having the following content:

Javascript is very simple language.

Create a JavaScript file named **main.js** having the following code:

```
var fs = require("fs");
var data = fs.readFileSync('input.txt');
console.log(data.toString());
console.log("Program Ended");
```



Open the **Node.js** command prompt and execute the following code.

node main.js

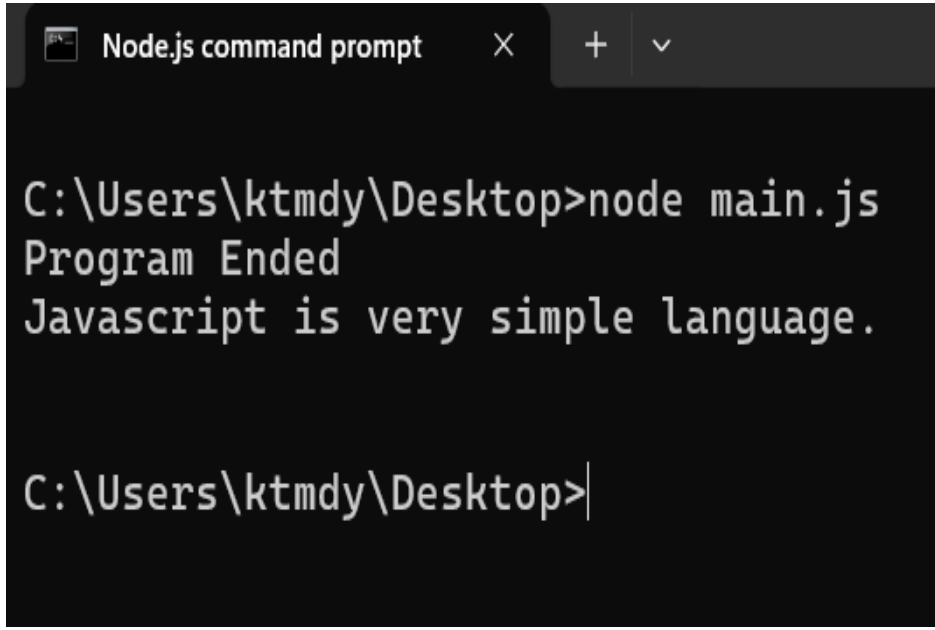
Non-Blocking Code with callbacks

Create a text file named **input.txt** having the following content:

Javascript is very simple language.

Create a JavaScript file named **main.js** having the following code:

```
var fs = require("fs");
 fs.readFile('input.txt', function (err, data) {
    if (err) return console.error(err);
    console.log(data.toString());
});
console.log("Program Ended");
```



Open the **Node.js** command prompt and execute the following code.

node main.js

Node.js Callbacks

- A Callback in Node.js is an asynchronous equivalent for a function.
- It is a special type of function passed as an argument to another function.
- Node.js makes heavy use of callbacks.
- Callbacks help us make asynchronous calls.
- All the APIs of Node are written in such a way that they support callbacks.

Node.js Callbacks

- Programming instructions are executed synchronously by default. If one of the instructions in a program is expected to perform a lengthy process, the main thread of execution gets blocked. The subsequent instructions can be executed only after the current I/O is complete. This is where callbacks come in to the picture.

- The callback is called when the function that contains the callback as an argument completes its execution, and allows the code in the callback to run in the meantime. This makes Node.js highly scalable, as it can process a high number of requests without waiting for any function to return results.

Node.js Callbacks

- **The syntax of implementing callback in Node.js is as follows −**

```
function function_name(argument, function (callback_argument){
        // callback body
})
```

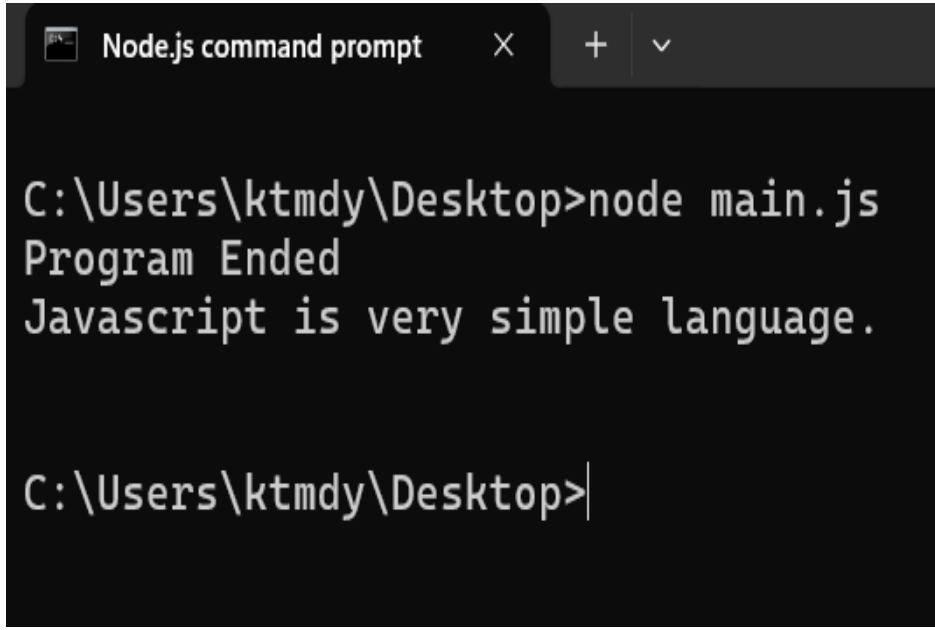
Non-Blocking Code with callbacks

Create a text file named **input.txt** having the following content:

Javascript is very simple language.

Create a JavaScript file named **main.js** having the following code:

```
var fs = require("fs");
 fs.readFile('input.txt', function (err, data) {
    if (err) return console.error(err);
    console.log(data.toString());
});
console.log("Program Ended");
```



Open the **Node.js** command prompt and execute the following code.

node main.js

Node.js Callbacks

- **You can also assign an arrow function as a callback argument.**
- **Arrow function in JavaScript is an anonymous function.**
- **It is also called as lambda function.**
- **The syntax of using arrow function as Node.js callback is as follows −**

```
function function_name(argument, (callback_argument) => {

   // callback body

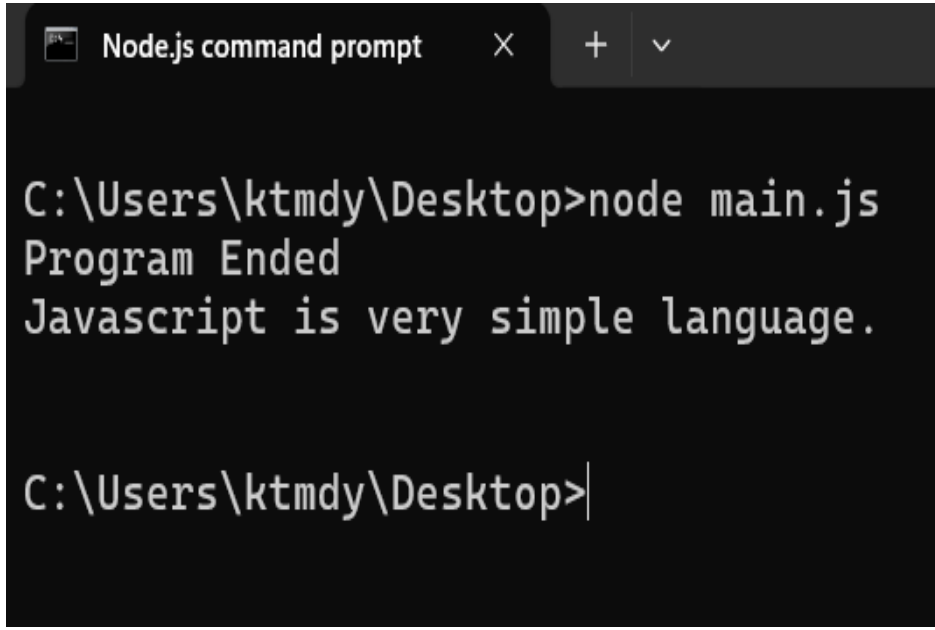
})
```

Create a text file named **input.txt** having the following content:

Javascript is very simple language.

Create a JavaScript file named **main.js** having the following code:

```
var fs = require("fs");
 fs.readFile('input.txt', (err, data)=> {
    if (err) return console.error(err);
    console.log(data.toString());
});
console.log("Program Ended");
```



Open the **Node.js** command prompt and execute the following code.

node main.js

Node.js Event Loop

- The event loop allows Node to perform non-blocking I/O operations despite the fact that JavaScript is single-threaded.
- It is done by assigning operations to the operating system whenever and wherever possible.
- When one of these operations is completed, the kernel tells Node.js, and the respective callback assigned to that operation is added to the event queue which will eventually be executed.

Node.js Event Loop

- An event loop is an endless loop, which waits for tasks, executes them, and then sleeps until it receives more tasks.
- The event loop executes tasks from the event queue only when the call stack is empty i.e. there is no ongoing task.
- The event loop allows us to use callbacks and promises.
- The event loop executes the tasks starting from the oldest first.

Node.js Event Loop Example

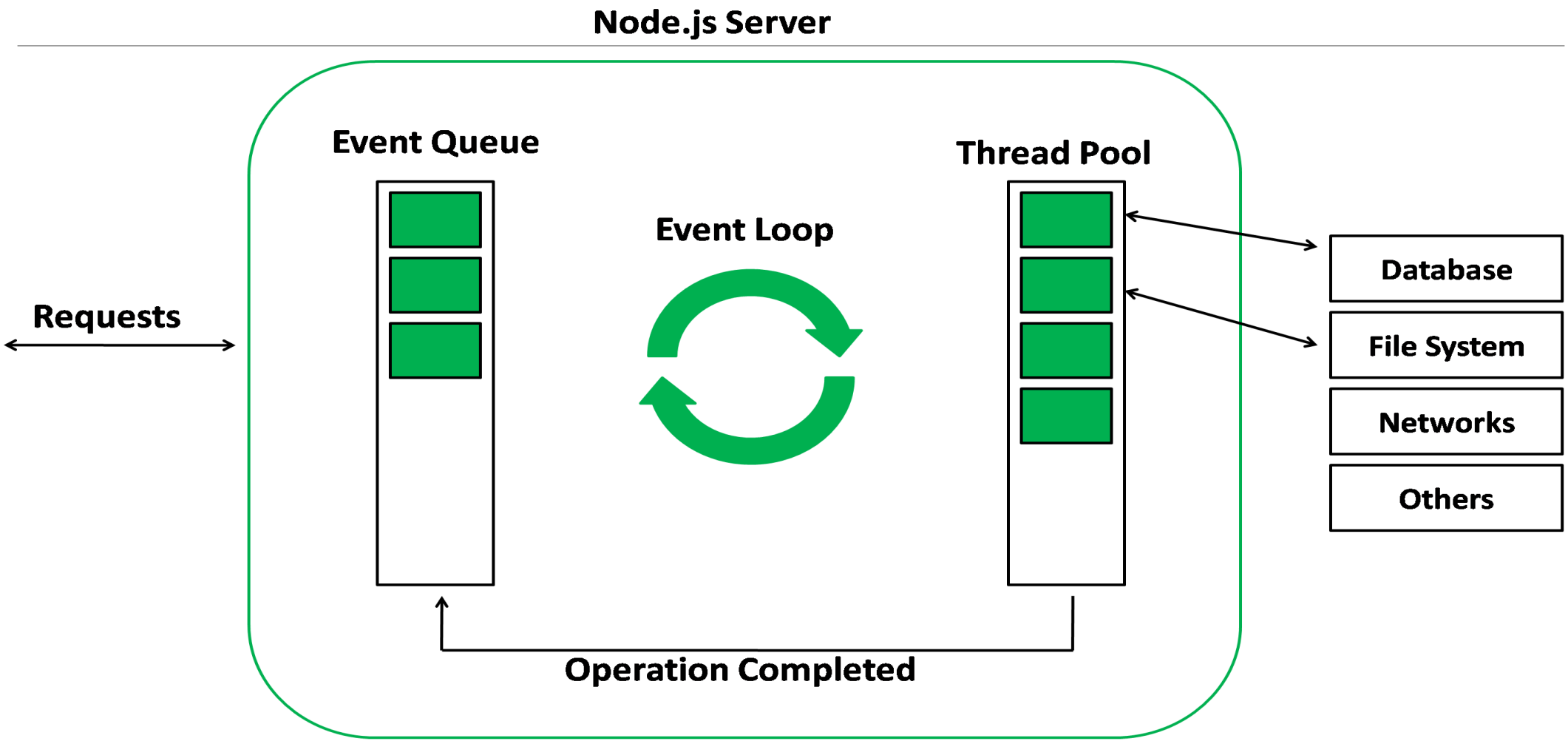
```
console.log("This is the first statement");
setTimeout(function(){
    console.log("This is the second statement");
                        }, 1000);
console.log("This is the third statement");
```

o/p - This is the first statement

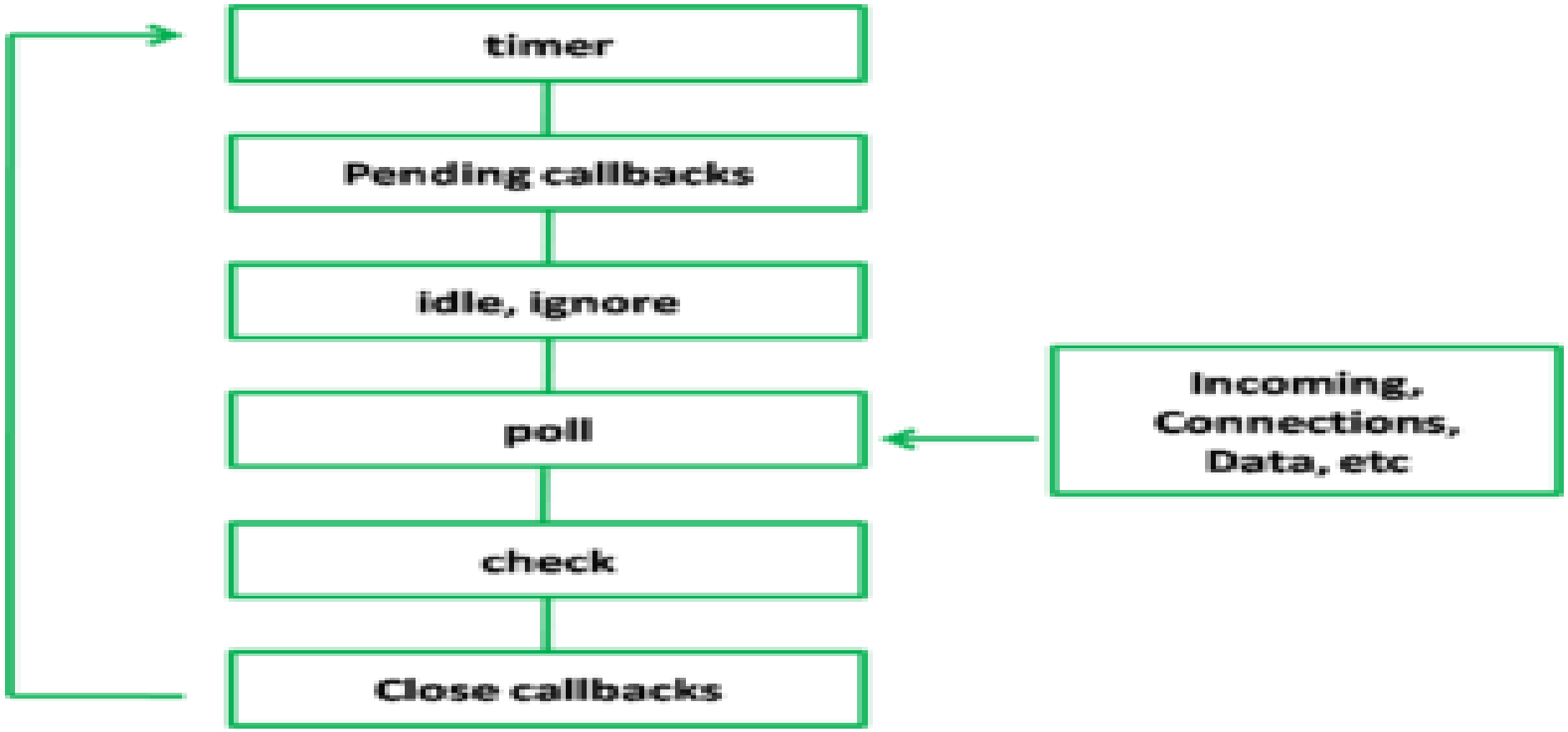
This is the third statement

This is the second statement

Node.js Event Loop



Node.js Event Loop Phases



Node.js Event Loop Phases

- **Timers**: This phase processes timers that have been set using setTimeout() and setInterval().

- **Pending Callbacks**: This phase processes any callbacks that have been added to the message queue by asynchronous functions.

- **Idle, Prepare:** The “idle” phase is a period of time during which the event loop has nothing to do and can be used to perform background tasks

- **Poll:** This phase is used to check for new I/O events and process any that have been detected.

- **Check:** This phase processes any setImmediate() callbacks that have been added to the message queue.

- **Close Callbacks:** This phase processes any callbacks that have been added to the message queue by the close event of a socket.

Node.js File System

- The Node.js API is a server-side programming technology.
- Hence, a Node.js application may be required to interact with the physical file system of the server.
- The Node.js API includes **fs module** that enables the developer to perform **read/write operations on disk files**.
- The fs module in Node.js provides **synchronous as well as asynchronous** methods for file handling.
- To use the filesystem functions, you need to import the fs module using the following syntax

var fs = require("fs")

Node.js File Writing

- Every method in the fs module has synchronous as well as asynchronous version.

- Asynchronous methods take the last parameter as the completion function callback and the first parameter of the callback function as error.

- For example, the **synchronous method** for writing data in a file is −

fs.writeFileSync(file, data[, options])

- On the other hand, its **asynchronous version** has the following syntax −

fs.writeFile(file, data[, options], callback)

- The asynchronous methods are non-blocking in nature as compared to the synchronous methods.

Node.js File Writing

```
const fs = require('fs');

var text = `Udemy is giving self learning content to teach the world in simple and easy way!!!!!`;

console.log("Writing synchronously");

fs.writeFileSync("input.txt", text);

console.log("Writing asynchronously");

fs.writeFile('input.txt', text, function (err) {
   if (err)
      console.log(err);
   else
      console.log('Write operation complete.');
});
```

Node.js File Reading

- For example, the **synchronous method** for reading data from a file is −

fs.readFileSync(fileName [,options])

- On the other hand, its **asynchronous version** has the following syntax −

fs.readFile(fileName [,options], callback)

Node.js File Reading

```
const fs = require('fs');

console.log("Reading synchronously");

data = fs.readFileSync("input.txt");

console.log(data.toString());

console.log("Reading asynchronously");

fs.readFile('input.txt', function (err, data) {
   if (err) {
      return console.error(err);
   }
   console.log("Asynchronous read: " + data.toString());
});

console.log('Read operation complete.');
```

input.txt

Udemy is giving self learning content
to teach the world in simple and easy way!!!!!

Node.js File Opening

- you can explicitly open a file, set the file opening mode indicating whether to use it for reading or writing, and then close the file.

- ***The open() method has the following signature −***

fs.open(path, flags[, mode], callback)

Parameters -

- ***path − This is the string having file name including path.***
- ***flags − Flags indicate the behavior of the file to be opened. All possible values have been mentioned below.***
- ***mode − It sets the file mode (permission and sticky bits), but only if the file was created. It defaults to 0666, readable and writeable.***
- ***callback − This is the callback function which gets two arguments (err, fd).***

Node.js File Opening

- The values of flag parameter are −

r - Open file for reading. An exception occurs if the file does not exist.

r+ - Open file for reading and writing. An exception occurs if the file does not exist.

rs - Open file for reading in synchronous mode.

rs+ - Open file for reading and writing, asking the OS to open it synchronously. See notes for 'rs' about using this with caution.

w - Open file for writing. The file is created (if it does not exist) or truncated (if it exists).

wx - Like 'w' but fails if the path exists.

w+ - Open file for reading and writing. The file is created (if it does not exist) or truncated (if it exists).

wx+ - Like 'w+' but fails if path exists.

a - Open file for appending. The file is created if it does not exist.

ax - Like 'a' but fails if the path exists.

a+ - Open file for reading and appending. The file is created if it does not exist.

ax+ - Like 'a+' but fails if the the path exists.

Node.js File Opening

```
const fd = fs.open('input.txt', 'w', (err, fd) => {

  if (err) {

    console.log(err);

    return;

  }

});
```

input.txt

Udemy is giving self learning content
to teach the world in simple and easy way!!!!!

Node.js File Opening and writing

```
const fs = require('fs');

// Open a file for writing

const fd = fs.open('input.txt', 'w', (err, fd) => {

   if (err) {

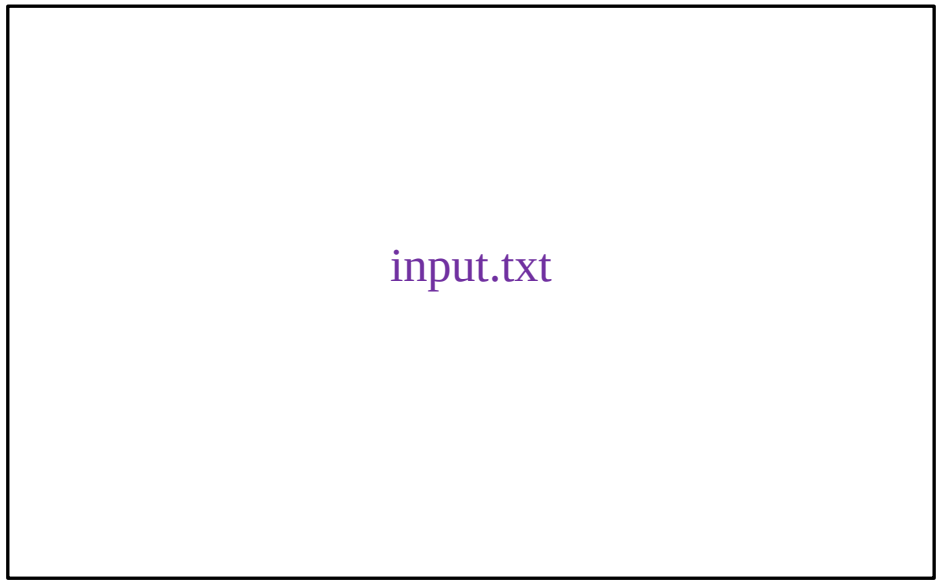
      console.log(err);

      return;

   }

   // Write some data to the file

   var data = `Udemy is giving self learning content to teach the world in simple and easy way!!!!!`;
```



```
fs.write(fd, data, (err) => {
  if (err) {
    console.log(err);
    return;
  }
```

```
// Close the file
    fs.close(fd, (err) => {
       if (err) {
          console.log(err);
          return;
       }
       console.log('The file was written successfully!');
    });
  });
});
```

Node.js File Opening and reading

```
var fs = require('fs');

fs.open('test.txt', 'r', function (err, fd) {

   if (err) {

      return console.error(err);

   }
```

input.txt

Udemy is giving self learning content
to teach the world in simple and easy way

```
var buffr = Buffer.alloc(1024);
```

```
fs.read(fd, buffr, function (err) {
 if (err) throw err;
     else
  console.log(buffr.toString());
});
```

```
// Close the opened file.
   fs.close(fd, function (err) {
      if (err) throw err;
   });
});
```

Node.js Get File information

- Following is the syntax of the method to get the information about a file −

fs.stat(path, callback)

Parameters

- path − This is the string having file name including path.
- callback − This is the callback function which gets two arguments (err, stats) where stats is an object of fs.Stats type which is printed below in the example.

Node.js Get File information

- Following is the syntax of the method to get the information about a file −

fs.stat(path, callback)

Parameters

- path − This is the string having file name including path.
- callback − This is the callback function which gets two arguments (err, stats) where stats is an object of fs.Stats type which is printed below in the example.

Node.js Get file information

```
var fs = require("fs");

console.log("Going to get file info!");

fs.stat('input.txt', function (err, stats) {
   if (err) {
      return console.error(err);
   }
   console.log(stats);
   console.log("Got file info successfully!");
  // Check file type
   console.log("isFile ? " + stats.isFile());
   console.log("isDirectory ? " + stats.isDirectory());
});
```

input.txt

Udemy is giving self learning content
to teach the world in simple and easy way

Node.js Closing a File

- Following is the syntax to close an opened file −

fs.close(fd, callback)

Parameters

- fd − This is the file descriptor returned by file fs.open() method.
- callback − This is the callback function No arguments other than a possible exception are given to the completion callback.

Node.js Deleting a File

- Following is the syntax of the method to delete a file −

fs.unlink(path, callback)

Parameters

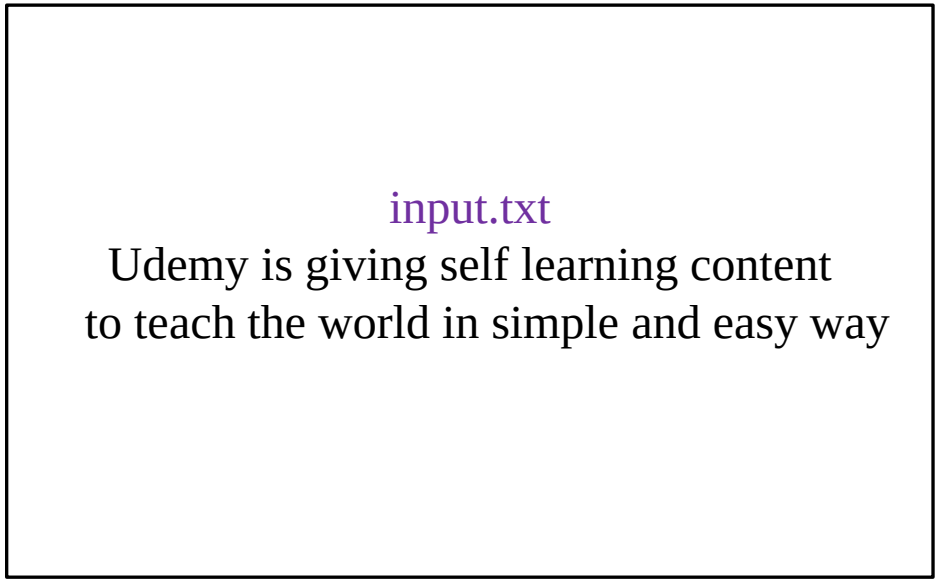
- path − This is the file name including path.
- callback − This is the callback function No arguments other than a possible exception are given to the completion callback.

Node.js Deleting a file

```
var fs = require("fs");

console.log("Going to delete an existing file");

fs.unlink('input.txt', function(err) {
   if (err) {
      return console.error(err);
   }
   console.log("File deleted successfully!");
});
```



Node.js Create a directory

- Following is the syntax of the method to create a directory −

fs.mkdir(path[, mode], callback)

Parameters

- path − This is the directory name including path.
- mode − This is the directory permission to be set. Defaults to 0777.
- callback − This is the callback function No arguments other than a possible exception are given to the completion callback.

Node.js Create a directory

```
var fs = require("fs");

console.log("Going to create directory /tmp/test");

fs.mkdir('/tmp/test',function(err) {

   if (err) {

      return console.error(err);

   }

   console.log("Directory created successfully!");

});
```

Node.js Read a directory

- Following is the syntax of the method to read a directory −

fs.readdir(path, callback)

Parameters

- path − This is the directory name including path.
- callback − This is the callback function which gets two arguments (err, files) where files is an array of the names of the files in the directory excluding '.' and '..'.

Node.js Read a directory

```
var fs = require("fs");

console.log("Going to read directory /tmp");

fs.readdir("/tmp/",function(err, files) {
   if (err) {
      return console.error(err);
   }
   files.forEach( function (file) {
      console.log( file );
   });
});
```

Node.js Remove a Directory

- Following is the syntax of the method to remove a directory −

fs.rmdir(path, callback)

Parameters

- path − This is the directory name including path.
- callback − This is the callback function No arguments other than a possible exception are given to the completion callback.

Node.js Remove a directory

```
var fs = require("fs");

console.log("Going to delete directory /tmp/test");

fs.rmdir("/tmp/test",function(err) {
   if (err) {
      return console.error(err);
   }
});
```

Global Objects

- Global objects in Node.js are **built-in objects**.

- The Node.js runtime is made up of a number of core modules.

- To incorporate the functionality of any of the core modules such as fs module, or http module (or any external module installed from npm – such as express module), you need to load the same with require() function.

- However, **some modules, functions, classes, variables etc. can be used directly in the JavaScript code, without loading them with require() functions.** They are called **Global objects. Let us take a look at the global classes, variables and functions in Node.js**

Global Classes – 1. Buffer Class

- Buffer class is a global class that can be accessed in an application without importing the buffer module.
- An object of buffer class stores raw data similar to an array of integers but corresponds to a raw memory allocation outside the V8 heap.
- You can create an uninitiated Buffer of 10 octets −

var buf = new Buffer(10);

Global Classes – 1. Buffer Class

- A buffer object can also be created from a given array −

---

var buf = new Buffer([10, 20, 30, 40, 50]);

- or from a given string −

var buf = new Buffer("Simply Easy Learning", "utf-8");

- You can perform different operations with a buffer such as read/write data, convert a buffer to JSON, concatenation of buffers etc.

Global Classes – 2. Console Class

- A Console class with methods such as console.log(), console.error(), and console.warn() that can be used to write to any Node.js stream.

- The global console can be used without calling require('node:console’).

```
console.log("Hello World");

console.error("Some error occurred");

console.warn("This is a warning message!");
```

Console Class

- **console.clear(): It is used to clear the console history**
- **console.count(): It is used to count the number of times a specific label has been called.**
- **E.g.**

```
const console = require('console');
  // Calling console.count()
console.count("a");
console.count("b");
console.count("a");
console.count("a");
console.count("a");
console.count("b");
```

Output

a: 1
b: 1
a: 2
a: 3
a: 4
b: 2

Console Class

- **console.info():** It is used to write a messages on console and it is an alias of console.log() method.

console.info("this is a sample info message!");

- **console.time():** It is used to starts a timer that is used to compute the time taken by a piece of code or function.

- The method console.timeEnd() is used to **stop the timer and output** the elapsed time in milliseconds to stdout.

Global Classes – 3. Process Class

- The process object is a global object, although it is defined in process module.
- The process object provides information on current process.
- With the help of a number of methods and properties associated with this object, it is possible to control the current process.

```
console.log('Process Architecture:'+process.arch);
console.log('Current working directory:'+ process.cwd());
console.log('Process PID: '+process.pid);
console.log('Process Platform: '+process.platform);
console.log('Process Version: '+process.version);
```

Global Functions – Timer function

- The **setTimeout()** function is used to run a callback after a specified duration in milliseconds.

- *E.g*

```
function printHello() {
   console.log( "Hello, World!");
}
// Now call above function after 2 seconds
setTimeout(printHello, 2000);
```

Global Functions – Timer function

- The clearTimeout() function is used to stop a timer that was previously created with setTimeout().

- The setInterval() function is used to run a callback repeatedly after a specified duration in milliseconds.

- *E.g*

```
function printHello() {
   console.log( "Hello, World!");
}
// Now call above function after 2 seconds
setInterval(printHello, 2000);
```

Global Variables – __filename

- The __filename represents the filename of the code being executed.

- This is the resolved absolute path of this code file.

- For a main program, this is not necessarily the same filename used in the command line. The value inside a module is the path to that module file.

- *E.g*

```
console.log( __filename );
```

o/p - D:\nodejs\main.js

Global Variables – __dirname

- The __dirname represents the name of the directory that the currently executing script resides in.

- *E.g*

```
console.log( __dirname );
```

o/p - D:\nodejs

Streams

- A stream is a **collection of data**.

- However, unlike an array or a string, the **entire data in a stream object is not stored at once in the memory**.
- Instead, a **single chunk of data from the stream is brought into the memory**, at a time.
- It makes streams more efficient.
- Node.js applications are **best suited for developing data streaming applications**.

Streams Types

Node.js processes four fundamental types of streams −

- **Writable** − streams to which data can be **written**.
- **Readable** − streams from which data can be **read**.
- **Duplex** − streams that are both **Readable** and **Writable**.
- **Transform** − Duplex streams that can **modify or transform the data** as it is written and read.

Streams

Each type of Stream is an **EventEmitter instance** and **throws several events** at different instance of times. For example, some of the commonly used events are −

- **data** − This event is fired when there is data is available to read.
- **end** − This event is fired when there is no more data to read.
- **error** − This event is fired when there is any error receiving or writing data.
- **finish** − This event is fired when all the data has been flushed to underlying system.

Streams

Each type of Stream is an **EventEmitter instance** and **throws several events** at different instance of times. For example, some of the commonly used events are −

- **data** − This event is fired when there is data is available to read.
- **end** − This event is fired when there is no more data to read.
- **error** − This event is fired when there is any error receiving or writing data.
- **finish** − This event is fired when all the data has been flushed to underlying system.

Readable Stream

- The file object acts as a stream from which **the data can be read in a chunk of specific size**.
- In the following example, we call **createReadStream()** function from fs module to read data from a given file.
- The on event of the **readable stream collects the file contents till the end event is fired**.

Readable Stream

```
var fs = require("fs");

var data = '';

// Create a readable stream

var readerStream = fs.createReadStream('input.txt');

// Set the encoding to be utf8.

readerStream.setEncoding('UTF8');

// Handle stream events --> data, end, and error

readerStream.on('data', function(chunk) {

   data += chunk;

});
```

```
readerStream.on('end',function() {
   console.log(data);
});

readerStream.on('error', function(err) {
   console.log(err.stack);
});

console.log("Program Ended");
```

Writable Stream

- The createWriteStream() function from fs module creates an object of writable stream.
- Its write() method stores the data in the file given to the createWriteStream() function as an argument.

Writable Stream

```
var fs = require("fs");

var data = `Udemy is giving self learning content`;

// Create a writable stream

var writerStream = fs.createWriteStream('output.txt');

// Write the data to stream with encoding to be utf8

writerStream.write(data,'UTF8');

// Mark the end of file

writerStream.end();
```

```
// Handle stream events --> finish, and error
writerStream.on('finish', function() {
   console.log("Write completed.");
});

writerStream.on('error', function(err){
   console.log(err.stack);
});

console.log("Program Ended");
```

Piping the Streams

- Piping is a mechanism where we provide the output of one stream as the input to another stream.
- It is normally used to get data from one stream and to pass the output of that stream to another stream.
- There is no limit on piping operations.
- Now we'll show a piping example for reading from one file and writing it to another file.

Piping the Streams

```
var fs = require("fs");

// Create a readable stream

var readerStream = fs.createReadStream('input.txt');

// Create a writable stream

var writerStream = fs.createWriteStream('output.txt');

// Pipe the read and write operations

// read input.txt and write data to output.txt

readerStream.pipe(writerStream);

console.log("Program Ended");
```

Chaining the Streams

- Chaining is a mechanism to connect the output of one stream to another stream and create a chain of **multiple stream operations**.
- It is normally used with piping operations.
- Now we'll use piping and chaining to **first compress a file and then decompress the same**.

Chaining the Streams

```
var fs = require("fs");

var zlib = require('zlib');

// Compress the file input.txt to input.txt.gz

fs.createReadStream('input.txt')

  .pipe(zlib.createGzip())

  .pipe(fs.createWriteStream('input.txt.gz'));


console.log("File Compressed.");
```

```
var fs = require("fs");

var zlib = require('zlib');

// Decompress the file input.txt.gz to input.txt

fs.createReadStream('input.txt.gz')

  .pipe(zlib.createGunzip())

  .pipe(fs.createWriteStream('input.txt'));


console.log("File Decompressed.");
```

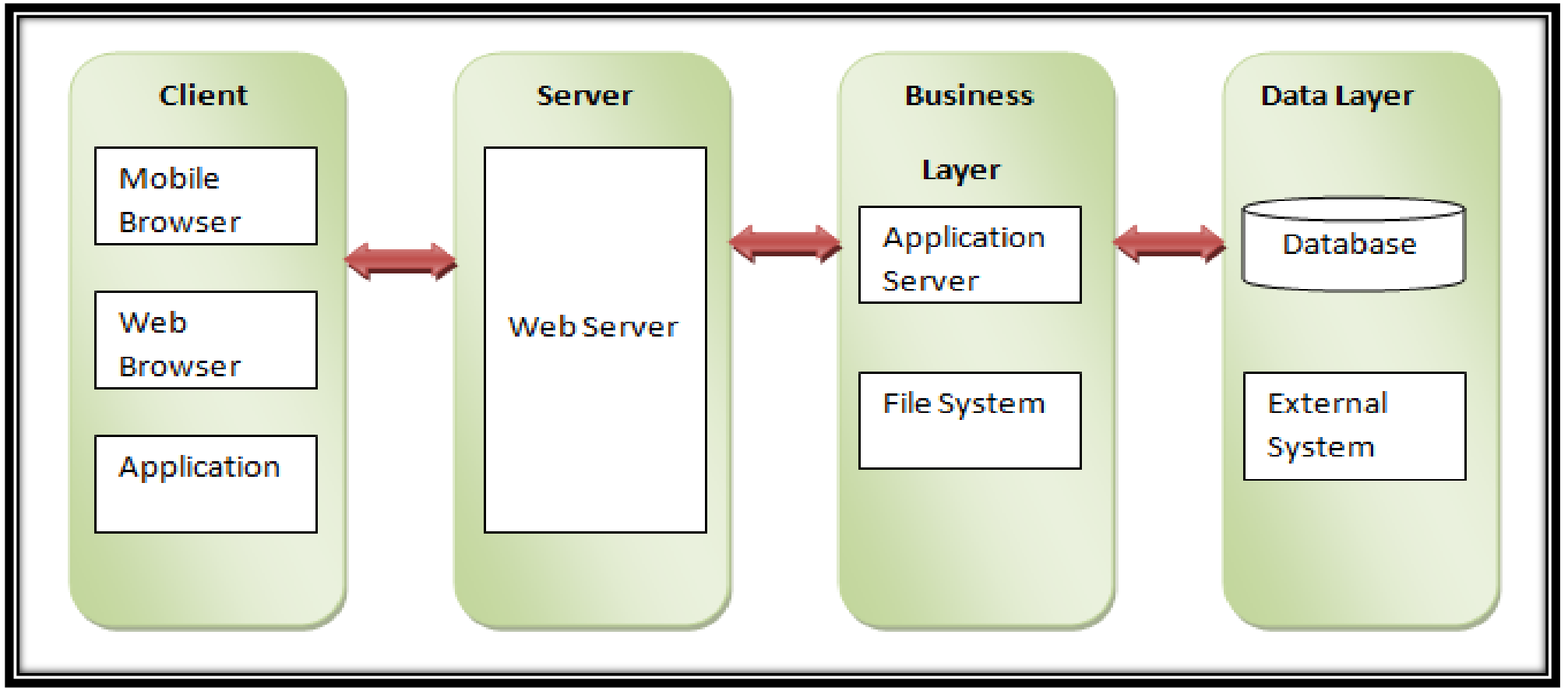
Web Module

- Web Server is a software program that **handles HTTP requests sent by HTTP clients** like web browsers, and **returns web pages in response** to the clients.
- Web servers usually **respond with html documents** along with images, style sheets and scripts.
- Most of the web server support **server side scripts using scripting language** or redirect to application server which perform the specific task of getting data from database, perform complex logic etc. and then sends a result to the HTTP client through the Web server.

Web Application Architecture

- **Client Layer:** The Client layer contains web browsers, mobile browsers or applications which can **make HTTP request to the web server**.
- **Server Layer:** The Server layer contains Web server which can **intercepts the request made by clients and pass them the response.**
- **Business Layer:** The business layer contains application server which is utilized by web server to **do required processing**. This layer interacts with data layer via data base or some external programs.
- **Data Layer:** The Data layer contains **databases or any source of data**.

Web Application Architecture



Creating Web Server using Node.js

```
var http = require('http');

var fs = require('fs');

var url = require('url');

  // Create a server

   http.createServer( function (request, response) {

   // Parse the request containing file name

   var pathname = url.parse(request.url).pathname;

   // Print the name of the file for which request is made.

   console.log("Request for " + pathname + " received.");
```

Creating Web Server using Node.js

```
// Read the requested file content from file system

fs.readFile(pathname.substr(1), function (err, data) {

   if (err) {

      console.log(err);

      // HTTP Status: 404 : NOT FOUND

      // Content Type: text/plain

      response.writeHead(404, {'Content-Type': 'text/html'});

   }
```

Creating Web Server using Node.js

```
else{

    //Page found

    // HTTP Status: 200 : OK

    // Content Type: text/plain

    response.writeHead(200, {'Content-Type': 'text/html'});

    // Write the content of the file to response body

    response.write(data.toString());

}
```

```
// Send the response body

    response.end();

  });

}).listen(8081);

// Console will print the message

console.log('Server running at http://127.0.0.1:8081/');
```

url: http://127.0.0.1:8081/index.html to be given on browser

Utility Modules

- There are several utility modules available in Node.js module library.
- These modules are very common and are frequently used while developing any Node based application.

Utility Modules

Sr.No.	Module Name & Description
1	**OS Module** Provides basic **operating-system related** utility functions.
2	**Path Module** Provides utilities for **handling and transforming file paths**.
3	**Net Module** Provides both **servers and clients as streams**. Acts as a network wrapper.
4	**DNS Module** Provides functions to **do actual DNS lookup** as well as to use underlying operating system name resolution functionalities.
5	**Domain Module** Provides ways to **handle multiple different I/O operations** as a single group.

OS Modules

- Node.js OS provides **operating-system related utility functions**.
- Node.js OS module includes a lot of practical utility methods that may be used to learn more about the computer system, that the OS module's software is now operating on.
- It can include details about hardware, including CPUs, endianness, the home directory, IP address, hostname, the operating system the program is running on, system uptime, details on the currently logged-in user, and more.

var os = require (“os")

OS Modules

Sr. No	Method	Description
1.	os.arch()	This method is used to fetch the operating system CPU architecture.
2.	os.cpus()	This method is used to fetch an array of objects containing information about each cpu/core installed: model, speed (in MHz), and times (an object containing the number of milliseconds the cpu/core spent in: user, sys, idle, etc.).
3.	os.networkinterfaces()	This method returns a list of network interfaces.
4.	os.freemem()	This methods returns the amount of free system memory in bytes.
5.	os.homedir()	This method returns the home directory of the current user.
6.	os.hostname()	This method is used to returns the hostname of the operating system.
7.	os.loadavg()	This method returns an array containing the 1, 5, and 15 minute load averages. The load average is a time fraction taken by system activity, calculated by the operating system and expressed as a fractional number.

OS Modules

Sr. No	Method	Description
8.	os.platform()	This method returns the operating system platform of the running computer i.e.'darwin', 'win32','freebsd', 'linux', 'sunos' etc.
9.	os.release()	This method returns the operating system release.
10.	os.tmpdir()	This method returns the operating system's default directory for temporary files.
11.	os.totalmem()	This method returns the total amount of system memory in bytes.
12.	os.type()	This method returns the operating system name. For example 'linux' on linux, and 'windows_NT' on windows 10.
13.	os.uptime()	This method returns the system uptime in seconds.
14.	os.userinfo([options])	This method returns a subset of the password file entry for the current effective user.

OS module Example

```
const os=require('os');

console.log("os.freemem(): \n",os.freemem());

console.log("os.homedir(): \n",os.homedir());

console.log("os.hostname(): \n",os.hostname());

console.log("os.loadavg(): \n",os.loadavg());

console.log("os.platform(): \n",os.platform());

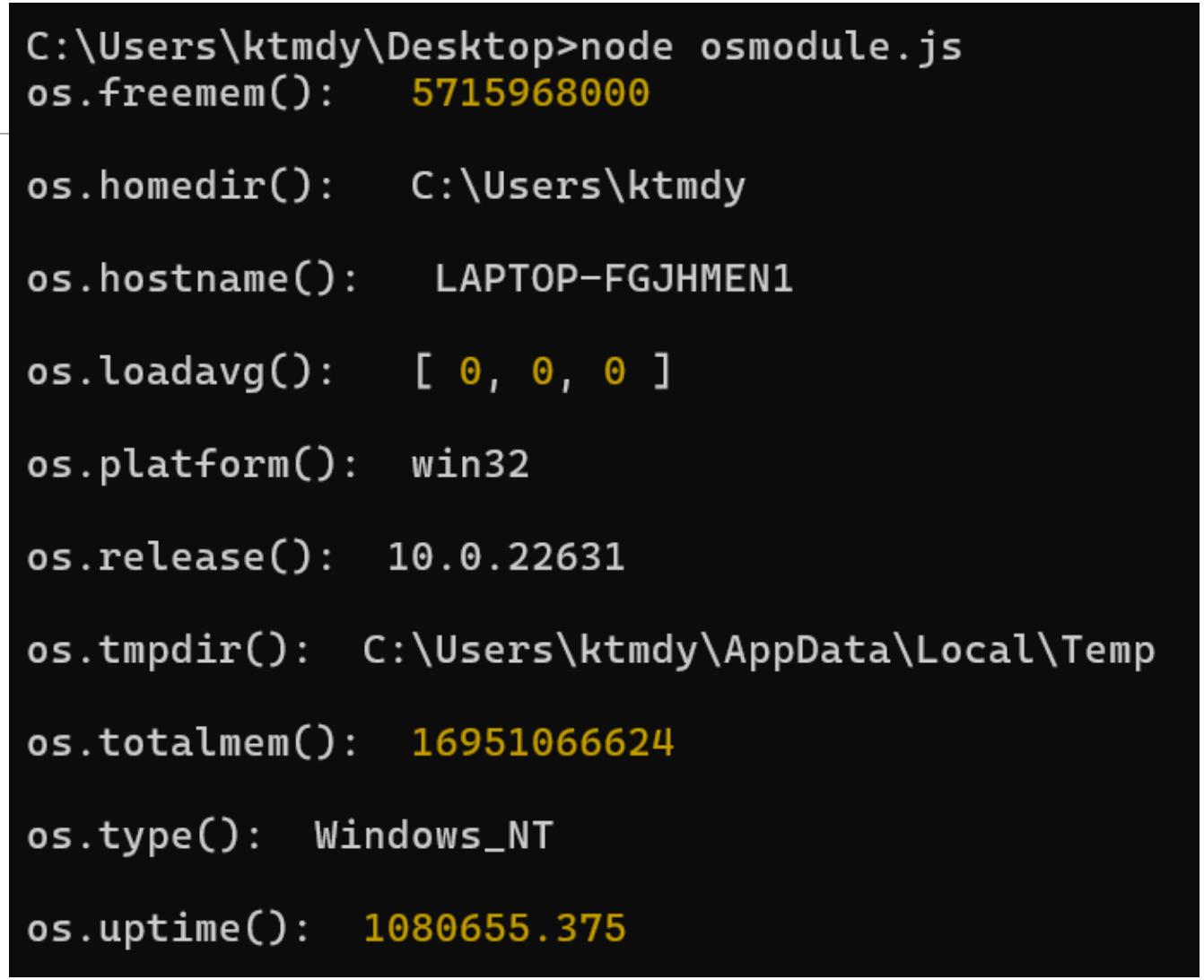
console.log("os.release(): \n",os.release());

console.log("os.tmpdir(): \n",os.tmpdir());

console.log("os.totalmem(): \n",os.totalmem());

console.log("os.type(): \n",os.type());

console.log("os.uptime(): \n",os.uptime());
```



Path Modules

- The Path module of Node.js offers utilities for **working with file and directory paths**.
- Depending on the type of operating system − LINUX or WINDOWS, the methods given in the Path module may yield varying results.
- Syntax:

```
var path =  require ("path")
```

Path Modules

Sr. No	Method	Description
1.	path.normalize(p)	It is used to normalize a string path, taking care of '..' and '.' parts.
2.	path.join([path1][, path2][, ...])	It is used to join all arguments together and normalize the resulting path.
3.	path.resolve([from ...], to)	It is used to resolve an absolute path.
4.	path.isabsolute(path)	It determines whether path is an absolute path. an absolute path will always resolve to the same location, regardless of the working directory.
5.	path.relative(from, to)	It is used to solve the relative path from "from" to "to".
6.	path.dirname(p)	It return the directory name of a path. It is similar to the unix dirname command
7.	path.basename(p[, ext])	It returns the last portion of a path. It is similar to the Unix basename command.

Path Modules

Sr. No	Method	Description
8.	path.extname(p)	It returns the extension of the path, from the last '.' to end of string in the last portion of the path. if there is no '.' in the last portion of the path or the first character of it is '.', then it returns an empty string.
9.	path.parse(pathstring)	It returns an object from a path string.
10.	path.format(pathobject)	It returns a path string from an object, the opposite of path.parse above.

Path module Example

```
var path = require("path");

// Normalization

console.log('normalization : ' + path.normalize('/sssit/apl//node/newfolder/tab/..'));

// Join

console.log('joint path : ' + path.join('/sssit', 'apl', 'node/newfolder', 'tab', '..'));

// Resolve

console.log('resolve : ' + path.resolve('path_example.js'));

// Extension

console.log('ext name: ' + path.extname('path_example.js'));
```

```
C:\Users\ktmdy\Desktop>node pathmodule.js
normalization : \sssit\apl\node\newfolder
joint path : \sssit\apl\node\newfolder
resolve : C:\Users\ktmdy\Desktop\path_example.js
ext name: .js
```

Net Modules

- Node.js net module is used to create both servers and clients.
- Syntax:

```
var net = require("net")
```

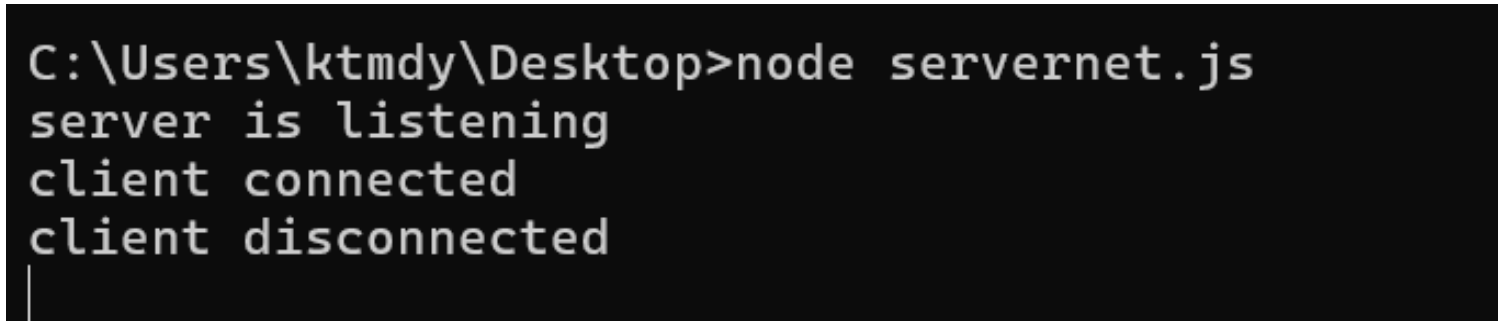
Net Modules

Sr. No	Method	Description
1.	net.createServer([options][, connectionListener])	Creates a new TCP server. The connectionListener argument is automatically set as a listener for the 'connection' event.
2.	net.connect(options[, connectionListener])	A factory method, which returns a new 'net.Socket' and connects to the supplied address and port.
3.	server.listen(port[, host][, backlog][, callback])	Begin accepting connections on the specified port and host.
4.	server.close([callback])	Finally closed when all connections are ended and the server emits a 'close' event.
5.	end	Emitted when the other end of the socket sends a FIN packet.
6.	error	Emitted when an error occurs. The 'close' event will be called directly following this event.

Net module Example

servernet.js

```
var net = require('net');
var server = net.createServer(function(connection) {
   console.log('client connected');
    connection.on('end', function() {
      console.log('client disconnected');
   });
      connection.write('Hello World!\r\n');
   connection.pipe(connection);
});
server.listen(8080, function() {
   console.log('server is listening');
});
```



Net module Example

clientnet.js

```
var net = require('net');

var client = net.connect({port: 8080}, function() {
   console.log('connected to server!');
});

client.on('data', function(data) {
   console.log(data.toString());
   client.end();
});

client.on('end', function() {
   console.log('disconnected from server');
});
```

```
C:\Users\ktmdy\Desktop>node clientnet.js
connected to server!
Hello World!

disconnected from server
```

DNS Modules

- Node.js DNS (Domain Name System) module is used to do actual DNS lookup as well as to use underlying operating system name resolution functionalities.
- The Node.js DNS module contains methods to get information of given hostname.
- Syntax:

```
var dns = require(“dns")
```

DNS Modules

Sr. No	Method	Description
1.	dns.lookup(hostname[, options], callback)	Resolves a hostname (e.g. 'google.com') into the first found A (IPv4) or AAAA (IPv6) record.
2.	dns.lookupService(address, port, callback)	Resolves the given address and port into a hostname and service using getnameinfo.
3.	dns.resolve(hostname[, rrtype], callback)	Resolves a hostname (e.g. 'google.com') into an array of the record types specified by rrtype.
4.	dns.reverse(ip, callback)	Reverse resolves an ip address to an array of hostnames.
5.	dns.getServers()	Returns an array of IP addresses as strings that are currently being used for resolution
6.	dns.setServers(servers)	Given an array of IP addresses as strings, set them as the servers to use for resolving.

DNS module Example

dnsmodule.js

```
const dns = require('dns');
dns.lookup('www.google.com', (err, addresses, family) => {
  console.log('addresses:', addresses);   //ip address
  console.log('family:',family);  //family i.e. IPv4 or IPv6
});
```

```
C:\Users\ktmdy\Desktop>node dnsmodule.js
addresses: 2404:6800:4009:826::2004
family: 6
```

Domain Modules

- Node.js domain module is used to **intercept unhandled error**.
- These unhandled error can be intercepted using **internal binding** or **external binding.** If errors are **not handled** at all, then they will **simply crash the Node application**.
- **Internal Binding** − Error emitter is executing its code **within the run method of a domain**.
- **External Binding** − Error emitter is added **explicitly to a domain using its add** method.
- Syntax:

```
const domain = require("domain");
```

- The **domain class** of domain module is used to provide functionality of routing errors and uncaught exceptions to the active Domain object.
- However, **this module is deprecated in newer versions** of Node.js and it's recommended to use modern error handling mechanisms such as **try-catch, promises or async-wait**

Domain Modules

Sr. No	Method	Description
1.	domain.run(function)	Run the supplied function in the context of the domain
2.	domain.add(emitter)	Explicitly adds an emitter to the domain. If any event handlers called by the emitter throw an error, or if the emitter emits an error event, it will be routed to the domain's error event, just like with implicit binding.
3.	domain.remove(emitter)	Removes domain handling from the specified emitter.
4.	domain.bind(callback)	The returned function will be a wrapper around the supplied callback function. When the returned function is called, any errors that are thrown will be routed to the domain's error event.
5.	domain.intercept(callback)	This method is almost identical to domain.bind(callback). However, in addition to catching thrown errors, it will also intercept Error objects sent as the first argument to the function.

Domain module Example

domainmodule.js

```
const domain = require('domain');

// Create a new domain
const d = domain.create();

// Handle errors that are captured within the domain
d.on('error', (err) => {
    console.error('Domain caught an error:', err);
});

// Use the domain to run code
d.run(() => {
    setTimeout(() => {
        console.log('This will run without errors.');
        // Simulate an error
        throw new Error('Something went wrong!');
    }, 1000);
    setTimeout(() => {
        console.log('Another operation running fine.');
    }, 1500);
});

console.log('Program is running...');
```

Domain module Example

```
C:\Users\ktmdy\Desktop>node domainmodule.js
Program is running...
This will run without errors.
Domain caught an error: Error: Something went wrong!
    at Timeout._onTimeout (C:\Users\ktmdy\Desktop\domainmodule.js:16:15)
    at listOnTimeout (node:internal/timers:581:17)
    at process.processTimers (node:internal/timers:519:7) {
  domainThrown: true
}
Another operation running fine.
```